Resultados del recomendador Two-Tower

CPIA — Motor de recomendación cinematográfica

Arnau Montull Uriel Martínez

19/10/2026 20:54

1. Resumen

En test (relevante = positivo) el Two-Tower supera a EASE en 9 de 9 métricas, promediando 1 semilla(s).

Recall@200: Two-Tower 0.6346, EASE 0.6030, popularidad 0.3332.

2. Datos y protocolo

Recomendador de películas *Two-Tower* entrenado con MovieLens 32M (200 948 usuarios, 23 335 películas, 31 7 M de valoraciones) y metadatos de TMDB. El reparto es **por usuario y por tiempo**: el 80 % más antiguo de cada usuario es entrenamiento, el 10 % siguiente validación y el último 10 % test. Todo lo que se aprende (media de cada usuario, popularidad, vocabularios) sale solo de entrenamiento. En validación el historial de entrada es train; en test, train y validación. En ambas particiones se excluye del ranking todo lo ya valorado. Las métricas se calculan sobre el catálogo completo (sin muestrear negativos), con una submuestra fija de 20 000 usuarios elegibles, la misma para los tres métodos. «Relevante» tiene dos definiciones: *positivo* (valoración $\geq \min(4, \bar{r}_u)$) y valoración ≥ 4. Baselines: popularidad (lo más visto en train) y EASE (Steck, 2019, $\lambda = 500$) sobre las 10 000 películas más populares.

3. Resultados principales

El Two-Tower se ha entrenado con 1 semilla(s): 42. Cada celda del Two-Tower es la media ± desviación típica muestral entre semillas; en negrita el mejor método.

Métrica	Popularidad	EASE	Two-Tower (media ± desv.)
Recall@10	0.0491	0.1149	**0.1291**
Recall@20	0.0839	0.1897	**0.2074**
Recall@50	0.1495	0.3297	**0.3521**
Recall@200	0.3332	0.6030	**0.6346**
NDCG@10	0.0497	0.1072	**0.1231**
NDCG@20	0.0598	0.1292	**0.1451**
NDCG@50	0.0796	0.1709	**0.1885**
NDCG@200	0.1273	0.2444	**0.2643**
Cobertura@20	0.0144	0.1634	**0.2041**
Novedad@20	8.3395	10.0466	**10.1403**

Tabla 1: Test, relevante = positivo.

Métrica	Popularidad	EASE	Two-Tower (media ± desv.)
Recall@10	0.0510	0.1178	**0.1338**
Recall@20	0.0868	0.1935	**0.2140**
Recall@50	0.1541	0.3351	**0.3604**
Recall@200	0.3401	0.6080	**0.6433**
NDCG@10	0.0482	0.1037	**0.1207**
NDCG@20	0.0589	0.1267	**0.1443**
NDCG@50	0.0796	0.1692	**0.1885**
NDCG@200	0.1268	0.2410	**0.2629**
Cobertura@20	0.0144	0.1634	**0.2041**
Novedad@20	8.3395	10.0466	**10.1403**

Tabla 2: Test, relevante = valoración ≥ 4.

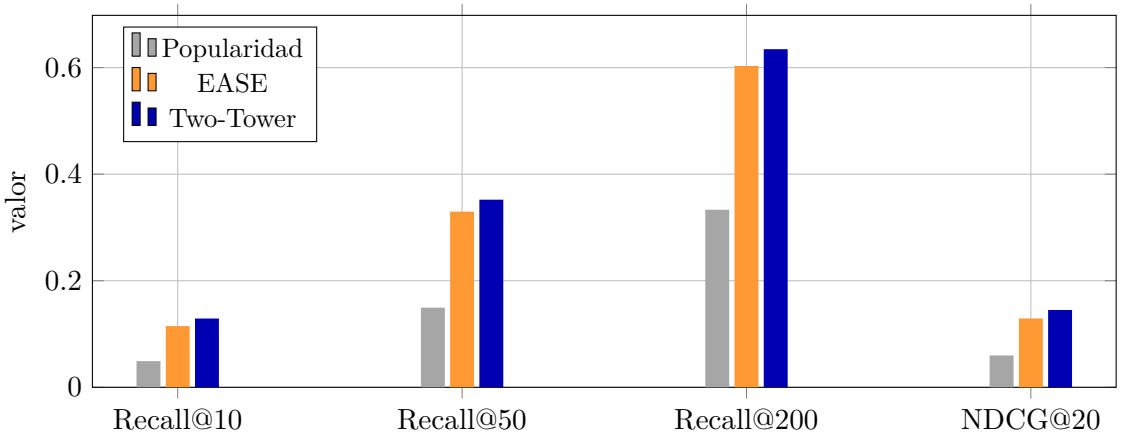


Figura 1: Test, relevante = positivo: tres métodos en las métricas principales.

Métrica	Popularidad	EASE	Two-Tower (media ± desv.)
Recall@10	0.0548	0.1193	**0.1359**
Recall@20	0.0936	0.2010	**0.2225**
Recall@50	0.1665	0.3539	**0.3797**
Recall@200	0.3561	0.6331	**0.6715**
NDCG@10	0.0518	0.1066	**0.1228**
NDCG@20	0.0638	0.1317	**0.1492**
NDCG@50	0.0862	0.1781	**0.1971**
NDCG@200	0.1355	0.2540	**0.2763**
Cobertura@20	0.0113	0.1597	**0.1946**
Novedad@20	8.3210	10.0078	**10.0811**

Tabla 3: Validación, relevante = positivo.

3.1. Variabilidad entre semillas

Semilla	`Recall@10`	`Recall@200`	`NDCG@20`	`Cobertura@20`
42	0.1291	0.6346	0.1451	0.2041

Tabla 4: Two-Tower por semilla en test (relevante = positivo).

3.2. Entrenamiento

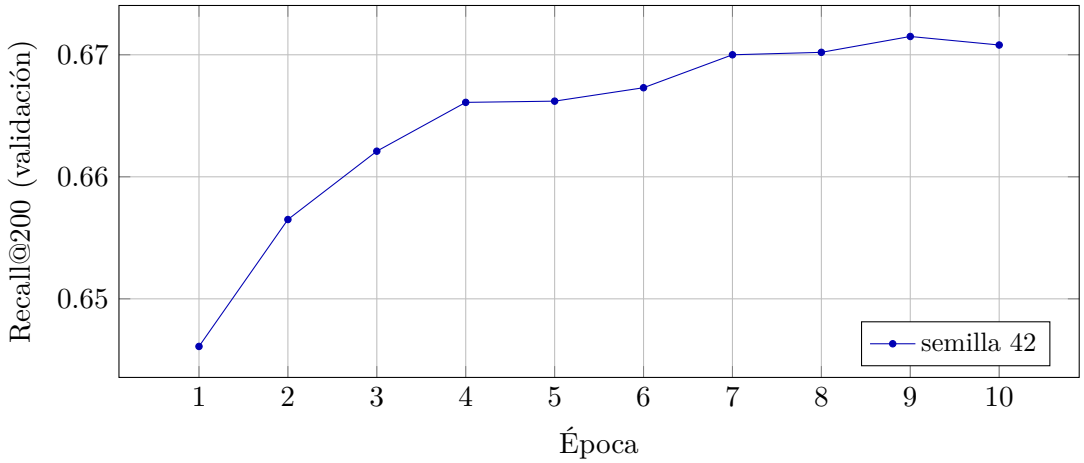


Figura 2: Recall@200 en validación por época, una curva por semilla.

4. Significación estadística

No se ha calculado el bootstrap de usuarios en esta ejecución.

5. Arranque en frío de usuarios

El experimento de arranque en frío no se ha ejecutado en esta versión del informe.

6. Validación con la watchlist

La watchlist son películas que el usuario marcó como «quiero ver»; el modelo no la usa como entrada. Es una señal débil (intereses, no gustos) y tiende a contener clásicos populares, por eso se compara con popularidad. Se excluyen las ya vistas, y solo se cuentan las emparejadas con el catálogo. No se ha evaluado la watchlist en esta ejecución.

7. Comprobaciones y pruebas

Comprobación	s42
determinismo	OK
exclusion c14	OK
finitud y normas	OK
id de peliculas frias	OK
invariancia orden historial	OK
metricas azar y oraculo	OK
numpy igual torch	OK

Tabla 5: Comprobaciones sobre datos reales, por checkpoint.

7.1. Latencia de inferencia (CPU, Python)

Un usuario (torre de usuario, puntuar las 23 335 películas y top-200 exacto): 3.63 ms. En lotes de 512: 0.458 ms por usuario. Calcular la torre de ítem completa: 83.7 ms (una vez por actualización del modelo).

8. Limitaciones

- EASE se calcula sobre las 10 000 películas más populares (la inversión densa de las 23 335 necesita unos 9 GB); con el catálogo completo podría mejorar algo.
- Las métricas se calculan sobre una submuestra fija de 20 000 usuarios, no sobre todos los elegibles.
- El mismo hiperparámetro $\tau = 0{,}05$ en todas las semillas: no se ha ajustado por semilla ni se ha barrido en el conjunto completo (solo en `dev`).
- La validación con la watchlist y las demos con perfiles de Letterboxd se hacen con dos usuarios: son ilustrativas, no estadísticas.

9. Reproducibilidad

Código y resultados en el repositorio `CPIA-recommender`. Entrenamiento: `python -m src.entrenar -muestra completo -device cuda -tau 0.05 -semilla N`. Evaluación: `python -m src.evaluar`. Comprobaciones: `python -m src.comprobaciones`. Este informe: `python -m src.informe_final -compilar`.

10. Glosario

Cada métrica, variable y término técnico del informe, tal como se usa en este proyecto. Ordenado por temas.

Datos

MovieLens 32M Conjunto público de valoraciones de películas (GroupLens): unos 32 millones de valoraciones de 0,5 a 5 estrellas de unos 200 000 usuarios. Es la fuente de las valoraciones.

TMDB The Movie Database: base de datos pública de películas. Aporta director, reparto, duración, palabras clave y sinopsis, que el modelo usa como «contenido» de cada película.

Letterboxd Red social de cine. Cada usuario puede exportar sus valoraciones (ratings.csv), películas vistas (watched.csv) y lista de pendientes (watchlist.csv). Se usa para la demo con perfiles reales.

Catálogo Las 23 335 películas que tienen al menos 20 valoraciones en MovieLens. Son las únicas que el modelo puede recomendar.

user_idx / item_idx Números enteros contiguos (0, 1, 2...) que identifican a cada usuario y a cada película dentro del código. Se usan en lugar de los identificadores originales (userId, movieId) porque indexan tablas y matrices.

movieId / tmdbId Identificador de una película en MovieLens / en TMDB. El catálogo guarda ambos, y el segundo permite emparejar películas de Letterboxd por la API de TMDB.

Partición (train / val / test) Reparto de las valoraciones de cada usuario por orden temporal: el 80 % más antiguo es train (entrenamiento), el 10 % siguiente val (validación, para decidir cuándo parar) y el último 10 % test (evaluación final, que nunca se usa para entrenar ni decidir).

Reparto por usuario y tiempo Cada usuario se reparte por separado y por fecha, de modo que el modelo nunca aprende de valoraciones posteriores a las que se le piden predecir.

Usuario elegible Usuario con al menos un positivo en train y al menos una película relevante en la partición que se evalúa. Solo ellos cuentan en las métricas, para no promediar usuarios sin nada que acertar.

Submuestra de 20 000 usuarios Para ahorrar tiempo, las métricas se calculan con 20 000 usuarios elegibles elegidos al azar con una semilla fija; son los mismos para todos los métodos, de modo que la comparación es justa.

media_u Media de las valoraciones de entrenamiento de un usuario. Mide si puntúa alto o bajo en general y sirve para decidir qué le gusta (un 4 es «poco» para quien da muchos 5).

Positivo (relevante «positivo») Valoración $r \geq \min(4, \text{media_u})$: lo que se considera que le gusta al usuario. La definición alternativa del informe es «rating ≥ 4», un umbral fijo.

Nota baja Valoración $r \leq \text{media_u} - 1$ que no es positiva. Se usa como señal de «no le gusta» (bolsa B).

Zona neutra Valoraciones que no son ni positivas ni notas bajas. Entran en el historial con peso reducido (0,25).

Peso de un ejemplo Importancia de un positivo de entrenamiento en la pérdida: $(1 + \max(0, r - \text{media_u})) \times 0{,}75$ si es de las primeras 24 horas del usuario, sin el 0,75 si no. Premia las valoraciones muy por encima de la media del usuario.

Bolsa A / bolsa B Resumen del historial de un usuario como dos «bolsas» de películas con pesos: la A reúne sus positivos y neutras (lo que le gusta o no le disgusta) y la B sus notas bajas (lo que no le gusta). El orden dentro de la bolsa no importa.

Historial y vista train / train_val Películas que se le dan al modelo como entrada para describir a un usuario. En validación es solo lo de train; en test, train más validación (vista train_val), que es lo realista.

Máscara de valorados (C14) Conjunto de películas que el usuario ya ha valorado. Se excluyen siempre de las recomendaciones al evaluar, y en el entrenamiento se descartan como negativos.

Película fría Película sin valoraciones en train. El modelo no ha podido aprender su identificador, así que su vector sale solo de su contenido (director, reparto, sinopsis...).

Usuario frío / arranque en frío Usuario que no estaba en el entrenamiento. Se le da una parte de su historia (80 %) y se mide si el modelo acierta lo que vio después (20 %). Comprueba que el modelo sirve para gente nueva sin reentrenar.

Watchlist Lista de películas que un usuario de Letterboxd marcó como «quiero ver». El modelo nunca la usa como entrada; se emplea para una evaluación sin etiquetas.

Emparejado exacto / por API Forma de reconocer una película de Letterboxd en el catálogo: por título y año normalizados (exacto) o, si falla, buscando en TMDB (API). El exacto no une títulos distintos de la misma película.

Modelo

Two-Tower (dos torres) Arquitectura de recomendación con dos redes: una convierte a cada película en un vector (torre de ítem) y otra a cada usuario en otro vector del mismo espacio (torre de usuario). La afinidad es el producto escalar de ambos.

Vector / embedding Lista de números (aquí 128) que representa a una película o a un usuario. Películas o usuarios parecidos quedan cerca en ese espacio.

d = 128 Dimensión de los vectores de usuario y de película.

Norma L2 / similitud coseno Los vectores se normalizan a longitud 1. Así el producto escalar es el coseno del ángulo entre ellos: 1 si apuntan igual, 0 si son perpendiculares. Es lo que mide cuánto encaja una película con un usuario.

Score Número que el modelo asigna a un par usuario–película: el producto escalar de sus vectores. Más alto = mejor encaje. Las recomendaciones son las películas de score más alto.

Torre de ítem Red que combina el contenido de la película (sinopsis, géneros, director, reparto, año, duración) y su identificador, y devuelve su vector.

Torre de usuario Red que toma la media ponderada de los vectores de las películas de las bolsas A y B del usuario, junto con indicadores (si la bolsa está vacía, cuántas películas) y devuelve su vector.

MiniLM (all-MiniLM-L6-v2) Modelo de lenguaje ya entrenado y congelado que convierte la sinopsis (más las palabras clave) en un vector de 384 números. No se reentrena.

EmbeddingBag Capa que convierte una lista de identificadores (los directores o los actores de una película) en la media de sus vectores aprendidos.

Dropout del ID (0,5) Durante el entrenamiento, en cada paso se anula con probabilidad 0,5 el identificador de cada película. Obliga al modelo a funcionar también solo con el contenido, necesario para películas sin valoraciones.

MLP / capa oculta / ReLU MLP (perceptrón multicapa): red de capas lineales con una función no lineal entre ellas. Aquí, dos capas con 256 neuronas ocultas. ReLU: la función que pone a 0 los valores negativos.

Pérdida contrastiva (softmax con negativos del lote) Función que el entrenamiento minimiza: para cada par usuario–película positivo, sube su score frente al de las demás películas del mismo lote, que actúan como ejemplos negativos.

logQ (corrección del sesgo de muestreo) Se resta a cada score el logaritmo de la popularidad de la película (log P(i)). Corrige que las películas populares aparecen más como negativos del lote y evita que el modelo aprenda solo popularidad.

Temperatura τ Parámetro que divide los scores antes del softmax. Con τ pequeña el modelo se concentra en separar lo mejor; con τ grande es más permisivo. Aquí $\tau = 0{,}05$.

Lote (batch) / paso / época Lote: grupo de ejemplos que se procesan a la vez (4 096). Paso: una actualización del modelo con un lote. Época: pasada por todos los ejemplos de entrenamiento (unos 3 700 pasos en el conjunto completo).

AdamW Algoritmo que ajusta los pesos del modelo para reducir la pérdida; Adam con regularización por decaimiento de pesos (weight decay 10^{-4}).

Tasa de aprendizaje (lr), calentamiento y decaimiento coseno lr: cuánto se mueven los pesos en cada paso (10^{-3} como máximo). Calentamiento: sube linealmente durante los primeros 200 pasos. Decaimiento coseno: baja suavemente hasta el 5 % al final.

Parada temprana (paciencia) Se evalúa en validación al final de cada época y se para si no mejora el Recall@200 durante 3 épocas seguidas. Evita sobreentrenar.

Checkpoint (mejor.pt / ultimo.pt) Archivo con los pesos del modelo y el estado del entrenamiento. mejor.pt es la época con mejor resultado en validación; ultimo.pt permite reanudar si el entrenamiento se corta.

Semilla Número que fija el azar del entrenamiento (inicialización de pesos y orden de los ejemplos). Cambiar de semilla mide cuánto depende el resultado de la suerte.

Métodos de referencia

Popularidad Método sin personalización: recomienda a todos los usuarios las películas con más positivos en train, excluyendo las ya valoradas. Es la referencia más básica.

EASE «Embarrassingly Shallow Autoencoder» (Steck, 2019): modelo lineal sin red neuronal que aprende, con una fórmula cerrada, qué películas suelen valorarse juntas. Muy fuerte para su sencillez. Aquí con $\lambda = 500$ y sobre las 10 000 películas más populares.

λ de EASE Regularización de EASE: cuanto mayor, más se suaviza lo aprendido. No confundir con λ del MMR.

Azar Recomendar películas al azar entre las no valoradas. Marca el mínimo esperable.

Oráculo Recomendador imaginario que conoce las respuestas. Su resultado es el techo posible y sirve para comprobar que la métrica está bien calculada.

Métricas

Top-k Las k películas con mejor score para un usuario, tras excluir las ya valoradas.

Recall@k De las películas relevantes de un usuario en esa partición, la fracción que aparece en su top-k. Va de 0 a 1; se promedia entre usuarios. Responde a «¿cuánto de lo que le gustaría encuentro?».

NDCG@k Normalized Discounted Cumulative Gain: premia acertar y que el acierto esté arriba. Cada acierto en la posición p suma $1/\log_2(p + 2)$; se divide por el máximo posible. De 0 a 1.

Techo de recall Máximo Recall@k alcanzable: media de min(k, nº de relevantes) / nº de relevantes. Si un usuario tiene más de k relevantes, ni un modelo perfecto llega a 1.

Cobertura@k Fracción del catálogo que aparece en el top-k de al menos un usuario evaluado. Cuanto mayor, más variedad global; una cobertura baja indica que se recomiendan siempre las mismas películas.

Novedad@k Media de $-\log_2 p(i)$ de las películas recomendadas, con p(i) la popularidad suavizada en train. Valores altos = recomienda películas poco populares.

Aciertos (hits@k), precisión@k, lift En la watchlist: aciertos = películas del top-k que están en ella; precisión = aciertos / k; lift = precisión dividida por la que tendría el azar (fracción de candidatos que están en la watchlist). Un lift de 30× es 30 veces mejor que el azar.

MMR (relevancia marginal máxima) y λ Reordenación que, en cada paso, elige la película con mejor equilibrio entre relevancia y diferencia con lo ya elegido: $\lambda \cdot$ relevancia $- (1 - \lambda) \cdot$ similitud máxima. $\lambda = 1$ mantiene el ranking original; menor λ da más variedad. Aquí $\lambda = 0{,}7$.

Estadística

Media ± desviación típica Media de un resultado entre semillas y su desviación típica muestral (dividida por $n - 1$). Indica cuánto cambia el resultado al cambiar solo el azar del entrenamiento.

Bootstrap de usuarios Técnica para medir la incertidumbre: se repite 2 000 veces el cálculo con usuarios remuestreados con reemplazo y se observa cuánto varía la diferencia entre dos métodos.

Intervalo de confianza (IC) del 95 % Rango donde cae el 95 % de los remuestreos del bootstrap. Si el IC de una diferencia no contiene el 0, la diferencia no se explica por qué usuarios entraron en la muestra.

Diferencia pareada Diferencia calculada usuario a usuario (el mismo usuario con los dos métodos) y luego promediada. Es más precisa que comparar medias sueltas.

Ingeniería

Fuga de datos (leakage) Error en el que el modelo usa, al entrenar o evaluar, información que no tendría en la práctica (por ejemplo, valoraciones posteriores a la que predice). Las comprobaciones del informe la descartan.

Invariante Propiedad que debe cumplirse siempre (por ejemplo, «las medias de usuario salen solo de train»). Se comprueba automáticamente sobre los datos reales.

Determinismo Dos ejecuciones con los mismos datos y el mismo modelo dan exactamente el mismo resultado.

CSR Formato compacto de matriz dispersa: guarda solo los elementos no nulos y dónde empieza cada fila. Así se guardan los historiales de millones de usuarios.

float32 / norma numérica Números decimales de 32 bits, los que usan el modelo y los ficheros exportados. Dos cálculos equivalentes pueden diferir en el último decimal (10^{-6}).

Latencia Tiempo de respuesta, en milisegundos (ms): lo que tarda en producir las recomendaciones para un usuario.

Colab / GPU T4 Entorno de Google donde se entrenó el modelo. La GPU T4 es la tarjeta gráfica gratuita que lo acelera: una época tarda unos 5,5 minutos.

Coherencia numpy / PyTorch Comprobación de que el vector de usuario calculado con PyTorch y con la reimplementación en numpy (la que usa la exportación a C) coincide hasta 10^{-6}.